Predictive Attack Behavioural Detection: A Pre-Execution Network Intrusion Early-Warning Framework using CUSUM Drift Monitoring, Ensemble Classification, and Explainable Signal Fusion

[Author Name][1], [Guide / Co-author Name][2]

[1] Student, Department of Artificial Intelligence and Machine Learning, [College Name], [University], [City], [State], India
[email]

[2] [Designation], Department of Artificial Intelligence and Machine Learning, [College Name], [University], [City], [State], India
[email]

Abstract. Most intrusion detection systems only tell a network administrator that an attack occurred once it has already begun executing, at which point there is very little that can be done about it. This paper presents Predictive Attack Behavioural Detection (PABD), a threat intelligence system built to catch the drift in network behaviour that happens in the minutes before an attack actually starts, instead of waiting for the attack itself to trip a signature. The system uses an Isolation Forest to score incoming traffic for anomalies, a CUSUM change point detector that watches ten behavioural metrics at once, a Random Forest classifier to work out what kind of attack is developing, and an LSTM network that estimates how much time is left before it hits. These four outputs are combined by a calibrated logistic fusion model into one risk score, and each detection is tagged with its MITRE ATT&CK tactic and technique and explained live using SHAP values, so an analyst can see exactly which behavioural features caused the alert. The whole pipeline was trained and tested on eight days of real CICIDS2017 traffic covering seven classes: Reconnaissance, Brute Force, DoS, DDoS, Botnet, Web Attacks and Normal traffic. On a chronologically split test set the Random Forest reaches 73% accuracy, and the fusion layer gets an AUC of 0.964, which is better than any single detector on its own (Random Forest confidence alone tops out at 0.930). The countdown LSTM's time to onset predictions have a mean absolute error of around 374 seconds on 50 held out real attack sequences. A black box evasion test also shows that an attacker can eventually push any of the evaluated samples across the decision boundary, but the amount of change needed ranges from 37% to 74% of the distance to normal traffic depending on the attack type, and a good part of that change touches features an attacker cannot realistically alter without breaking the attack itself. The CUSUM detector's false positive behaviour on unseen benign traffic is reported alongside the results rather than left out, since no signature based IDS was available for a direct side by side comparison.

Keywords: Network Intrusion Detection · Predictive Security Analytics · CUSUM Change-Point Detection · Ensemble Learning · Explainable AI · MITRE ATT&CK · Adversarial Robustness

1 Introduction

1.1 The Case for Predictive Intrusion Detection

Networks today carry far more traffic than they did even five years ago, and the attacks aimed at them have grown just as fast in both scale and sophistication. Most of the intrusion detection systems (IDS) protecting these networks, whether rule based tools like Snort and Suricata or machine learning classifiers trained on flow features, share the same basic weakness: they only recognise an attack once its signature is fully formed, which is usually well after the attacker has already gotten what they came for. Machine learning has pushed classification accuracy up considerably in recent years (Ferrag et al., 2020), and ensemble methods in particular handle the messy, high dimensional feature spaces that network flow data tends to produce. Even so, almost all of this work still treats detection as a single snapshot decision, is this traffic an attack or not, rather than looking at the traffic leading up to an attack as a signal in its own right.

The idea behind this work is that a lot of attack types show a build up before they actually do any damage. A port scan slowly widens its coverage. A brute force login attempt shows more and more failed authentications before it eventually gets one right. A denial of service attack ramps its connection rate up before it floods its target. If a system can pick up on this kind of build up reliably, it buys a defender time that a detector waiting for a full signature simply cannot give them.

1.2 Problem Statement and Proposed Solution

There is plenty of research answering "is this an attack," but far less on the follow up question of whether traffic is heading toward becoming one, and how much time is left. Change point detection methods such as CUSUM have been used for decades in industrial monitoring to pick up exactly this kind of drift (Farkas, 2016), but bringing CUSUM into network security has a well known problem attached to it: it is very sensitive to how the baseline and threshold are set, and a badly tuned detector ends up producing so many false alarms that nobody trusts it. This tracks with a point made repeatedly in the security literature, that most alerts raised by conventional network IDS turn out to be false positives.

PABD (Predictive Attack Behavioural Detection) is built around this gap. It does not try to replace classification based detection with CUSUM, but instead runs a tuned, multi metric CUSUM layer alongside an Isolation Forest anomaly scorer, a Random Forest classifier and an LSTM time to onset estimator, with all four combined through a fusion layer at the end. The main contributions of this paper are:

- A ten metric CUSUM early warning detector with a reset mechanism added to stop it sticking after a single spike, checked against a flow fraction threshold baseline for lead time on real traffic.
- A time to attack estimator trained on real pre attack sequences pulled from the actual CICIDS2017 capture timeline, not on synthetic data.
- A fusion layer that takes the Isolation Forest, CUSUM and Random Forest outputs and turns them into one risk score that ranks threats better than any of the three do individually.
- A live SHAP explanation and MITRE ATT&CK tag attached to every alert, computed for that specific window instead of a single fixed importance ranking for the whole model.
- An adversarial evasion test that measures how much an attacker would have to change their traffic to slip past the classifier, and a check on which of those changes are actually realistic for an attacker to make.
- An honest account of where the system's CUSUM layer struggles, including its false positive rate on traffic it was never tuned on, and which attack classes the real dataset simply does not have enough examples of.

The rest of the paper is laid out as follows. Section 2 covers related work on ML based intrusion detection and change point methods. Section 3 lays out the system architecture. Section 4 covers the dataset and methodology. Section 5 goes through the results. Section 6 wraps up with future work.

2 Background and Related Work

Large labelled traffic datasets and better ensemble and deep learning tools have opened the door to intrusion detection that looks at behaviour instead of just matching signatures. This section places PABD against that backdrop.

2.1 Machine Learning for Network Intrusion Detection

Random Forest and gradient boosted trees have both been applied heavily to flow data from datasets like CICIDS2017, usually with good accuracy on the bigger attack classes, though class imbalance and quirks specific to the dataset keep coming up as a problem (Sharafaldin et al., 2018). Panigrahi and Borah (2018) pointed out that CICIDS2017 has known issues in how its flows were constructed and labelled, and that anyone using it needs to account for that instead of treating it as ground truth. Isolation based anomaly detectors have also proven useful for flagging odd flows without needing labelled attack data to begin with (Liu et al., 2008), which is the reason this work uses one as a first, unsupervised layer before the supervised classifier runs.

2.2 Change-Point Detection and Early-Warning Systems

CUSUM and its variants have been used for network anomaly detection specifically because they catch gradual drift instead of one off spikes (Tartakovsky et al., 2006). A recurring problem in this line of work is that a plain CUSUM detector gets confused by seasonal or cyclical traffic and mistakes it for drift, and adaptive baseline or wavelet filtered versions cut down the resulting false alarms considerably (Farkas, 2016). This work follows a similar adaptive baseline approach, and on top of that fixes a separate issue found during development: once an unmitigated CUSUM accumulator gets tripped by one large but isolated spike, it can stay falsely triggered for a long stretch of windows afterward unless it is given a way to reset once things go back to normal.

2.3 Explainability and Adversarial Considerations in Intrusion Detection

SHAP based explanations have been used before to make intrusion detection decisions auditable for a security analyst (Lundberg and Lee, 2017), but most of these setups compute one global importance ranking for the whole model and stop there, which is not much help when an analyst is staring at one specific alert trying to work out why it fired. Adversarial robustness for network intrusion classifiers has also gotten far less attention than it has in image classification, even though a network attacker, unlike a static image, is actively trying to dodge detection. This work tries to close both gaps at once: it attaches a per alert SHAP explanation instead of a global one, and it runs an adversarial evasion test against the trained classifier on real attack traffic, which is not something the surveyed literature does together in one system.

3 System Architecture

The system watches live network traffic, picks up on behavioural drift before an attack happens, classifies the attack type once there is enough evidence, works out how much time is probably left, and puts all of that in front of an analyst in a form that can actually be checked and trusted. Figure 1 shows the overall data flow.

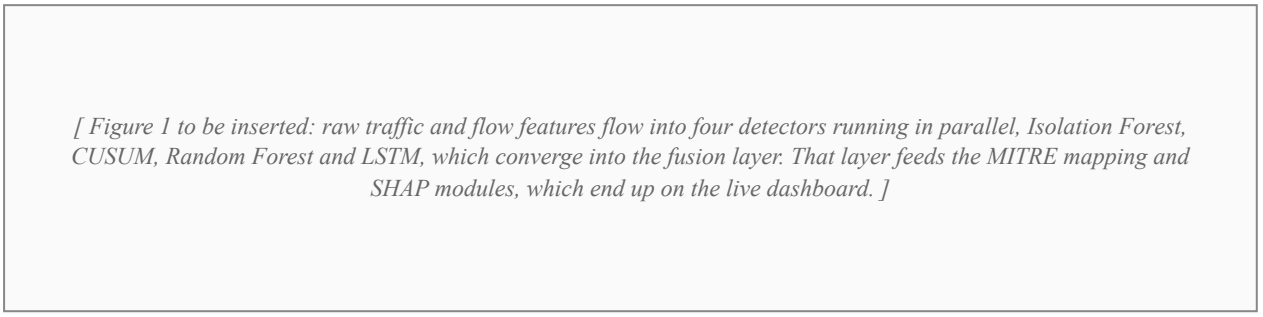


Figure 1: PABD System Architecture

The system has four detection layers working together, plus two more layers that add context on top.

The **anomaly layer** is an Isolation Forest trained only on benign traffic. It gives every window a continuous anomaly score without needing labelled attack data to be trained on, which is the point of using it here: it can still pick up on something odd even if that particular attack pattern was never in the labelled training set.

The **drift layer** runs a CUSUM detector separately on each of ten behavioural features (connection frequency, port entropy, failed authentication rate, packet size distribution, protocol concentration, burst activity, SYN flag rate, flow duration variability, mean byte rate, and the ratio of downstream to upstream traffic) and raises an alert once a set number of those ten cross their drift threshold at the same time. This is what gives the system its early warning property, since it is picking up on a gradual build up instead of waiting for a full signature to show itself.

The **classification layer** is a Random Forest trained on the same ten features, which works out what type of attack is developing once the drift layer has already fired.

The **temporal layer** is an LSTM trained on sequences of ten windows leading up to a real, confirmed attack, and it outputs a rough estimate in seconds of how much time is left before the attack actually hits.

All four of these feed into the **fusion layer**, a calibrated logistic model that takes the Isolation Forest score, the CUSUM breach count and the Random Forest's confidence and turns them into one combined risk score.

On top of that sits the **intelligence layer**: every alert gets a MITRE ATT&CK tactic and technique tag based on what it was classified as, and a live SHAP explanation computed specifically for that alert's own feature values, not a single fixed ranking reused across every alert.

4 Methodology

4.1 Dataset Details

The system is trained and tested on real traffic taken from eight days of the CICIDS2017 dataset (Sharafaldin et al., 2018), which covers benign traffic along with Reconnaissance, Brute Force, DoS, DDoS, Botnet and Web Attack traffic. Raw flow records are grouped into fixed 120 second windows, and the ten features described in Section 3 are computed for each window. Each window is labelled with whichever attack type makes up most of it, and a continuous attack fraction is kept alongside that label so lead time can be worked out later. Once this is done, the dataset has 861 labelled windows across seven classes: Normal (588), Botnet (90), Brute Force (66), DoS (51), Web Attacks (36), Reconnaissance (19) and DDoS (11). One day of purely benign traffic is kept completely separate and used only to check the false positive rate on traffic the system was never tuned against, covered in Section 5.

Two attack types that show up in the raw capture, Infiltration and Lateral Movement, were left out of the trained classifier on purpose. Infiltration only has 36 raw flow records in the whole capture, which is not enough to train a class on, so it is looked at separately as a small case study instead. Lateral Movement does not appear at all in the CICIDS2017 days used here, so it only shows up in the dashboard's demo mode and no accuracy number is claimed for it anywhere in this paper.

4.2 Behavioural Feature Engineering

Six of the ten features (connection frequency, port entropy, failed authentication rate, packet size distribution, protocol concentration, burst activity) come straight from flow level aggregates in each window. The other four were added on top because the first six do not capture certain signatures well: SYN flag rate for scanning behaviour, flow duration coefficient of variation for the short, similar looking flows a scan produces, mean flow byte rate for volumetric floods, and the ratio of downstream to upstream traffic for the directional imbalance seen in exfiltration and command and control traffic. When these four are compared against the original six in an ablation test, they come out as the four most important features by Random Forest importance, and adding them improves both classification accuracy and CUSUM detection, which is covered in Section 5.

4.3 CUSUM Drift Detection with Baseline Adaptation and Reset Correction

For each feature f, an accumulator s_f is kept and updated at every window as

$$s_f \leftarrow max(0,\ s_f + (x_f - \mu_f - k\sigma_f))$$

where x_f is the value observed, μ_f and σ_f are the mean and standard deviation of that feature under benign traffic, and k is a slack term. A window is flagged once s_f goes above $h\sigma_f$ for at least N out of the ten features at once. The values of k, h and N come from a grid search on held out labelled data, not from guesswork.

A reset was added on top of this: once a feature sits inside a tolerance band around its baseline for enough consecutive windows in a row, its accumulator gets forced back to zero. This was added after testing showed that a single large but one off spike could leave the accumulator stuck above threshold for a long stretch of windows afterward, since it only comes back down by the slack term $k\sigma_f$ each step regardless of how big the original spike was.

4.4 Time-to-Onset Estimation

The LSTM is trained on sequences of ten real, consecutive windows taken straight from the CICIDS2017 timeline, each one labelled with how many windows are actually left before the next attack starts, scaled against a fixed lookback horizon. This is a deliberate choice over training on synthetically generated pre attack sequences: since these sequences come from real attacks and not a generator, the error reported for this model reflects how hard the problem really is on live traffic.

4.5 Fusion of Detection Signals

The Isolation Forest score, the CUSUM breach count and the Random Forest's top class probability are fed into a calibrated logistic regression model, trained and tested on a chronological split kept separate from the split used to tune the individual detectors earlier. The output of that logistic model is what the system actually uses as its risk score, not any one of the three inputs on their own.

4.6 Adversarial Evasion Methodology

The Random Forest is a tree ensemble, so it is not differentiable and gradient based adversarial methods do not apply to it directly. A black box boundary search is used instead: for each real attack sample, a straight line is drawn in scaled feature space from that sample to the centroid of the benign class, and a binary search along that line finds the smallest step needed to flip the prediction to benign. This step size is reported both as a raw number and as a fraction of the full distance to the benign centroid, along with a judgement call on each feature about whether an attacker could actually make that change without the attack stopping being an attack. An attacker can slow down their request rate easily enough, for instance, but cannot flip the direction of a denial of service flood without simply not flooding anything.

Algorithm 1 PABD Predictive Detection Pipeline

– **Input:** Live network flow stream, benign-traffic baseline statistics (μ, σ), trained Isolation Forest IF, trained Random Forest RF, trained countdown LSTM, trained fusion model FM, CUSUM parameters (k, h, N).

– **Output:** Per-window anomaly score, drift-breach count, attack classification with confidence, estimated time-to-onset, composite risk score, MITRE tactic/technique tag, per-alert SHAP explanation.

1. Aggregate incoming flow records into a 120-second observation window *w* and compute the ten behavioural features $x = (x_1, \ldots, x_{10})$.
2. Compute the anomaly score $a = -\text{IF.score_samples}(x)$.
3. For each feature *f*, update the CUSUM accumulator: $s_f \leftarrow \max(0, s_f + (x_f - \mu_f - k\sigma_f))$, applying the reset correction of Section 4.3.
4. Count $b = |\{ f : s_f > h\sigma_f \}|$.
5. **If** $b \geq N$ **and** no alert has yet been raised for the current episode: raise a drift alert and record the current window index.
6. **If** a drift alert is active and ten consecutive windows are available: classify $\hat{y} = \text{RF}(x)$ with confidence $p = \max(\text{RF.predict_proba}(x))$, and compute the countdown estimate $\hat{t} = \text{LSTM}(x_{w-9}, \ldots, x_w) \times T_{\text{horizon}}$.
7. Compute the composite risk score $r = \text{FM}(a, b, p)$.
8. Retrieve the MITRE ATT&CK tactic and technique corresponding to $\hat{y}$.
9. Compute SHAP values $\varphi = \text{SHAP}(\text{RF}, x)$ for the predicted class $\hat{y}$, and rank features by $|\varphi|$.
10. Emit $(a, b, \hat{y}, p, \hat{t}, r, \text{tactic}, \text{technique}, \varphi)$ to the analyst dashboard.

4.7 Working of the Detection Pipeline

The drift layer and the anomaly layer run on every window independently, so a window can get flagged by the Isolation Forest without triggering a CUSUM breach, and the other way round too. The classification and temporal layers only kick in once the drift layer has already raised an alert, since trying to name an attack type or predict a countdown before there is enough drift evidence would not mean much. Once something is classified, the SHAP and MITRE layers work off that one window's own feature values, so what shows up on screen is specific to the traffic that actually caused the alert instead of a fixed summary computed once and reused everywhere. That

means an analyst goes from a raw anomaly signal to a classified, explained, time bounded threat without having to manually piece any of it together.

5 Discussion and Results

The framework is implemented in Python, with the Isolation Forest, Random Forest, and fusion layer built on scikit-learn, the countdown LSTM built on TensorFlow/Keras, explanation generation performed with the SHAP library, and the live dashboard built with Dash/Plotly.

5.1 Classification Performance

On a chronological 70/30 split of the 861 window dataset, the Random Forest gets an overall accuracy of 73%. Performance is not even across classes, which is more or less expected with the class imbalance real attack traffic tends to have. Botnet traffic, which mostly looks like a low volume, persistent connection pattern that is not that different from normal background noise, is the weakest class at an F1 score of 0.39, while classes with a more obvious volumetric signature do a lot better. DDoS gets a perfect score on the held out split, but that is only across two test set samples, so this number is not being treated as a real measure of how well the model handles DDoS in general, just a data point that comes with a very small sample size attached.

5.2 Drift Detection and the Precision-Recall Trade-off

Figure 2 shows precision, recall and false positive rate as the breach count threshold N is swept, checked both against the six attack days used for tuning and against a completely separate, purely benign day.

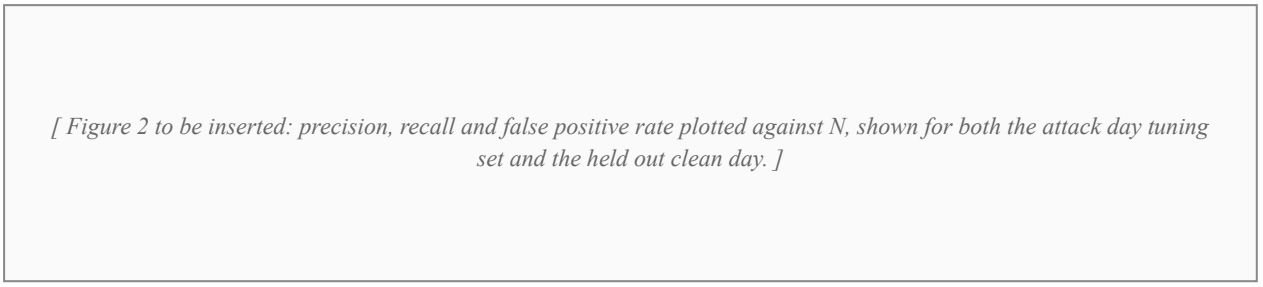


Figure 2: CUSUM Operating-Point Sweep

Table 2: CUSUM detector operating points, with and without the baseline-adaptive reset correction

N (of 10)	Precision	Recall	FPR (attack days)	FPR (clean day)
2	0.410	0.722	0.483	0.631–0.730
3	0.372	0.425	0.333	0.504
4	0.365	0.333	0.269	0.279
5	0.378	0.205	0.156	0.184
6	0.132	0.026	0.078	0.000

N=2 gives the best F1 score and picks up 72% of attacks in the tuning data, but its false positive rate on the clean held out day is 63 to 73%, so at that setting it cannot really be told apart from noise on traffic it was not tuned on. That lines up with what the wider IDS literature already says about most change point and threshold alerts turning out to be false positives, and it is left in the results as it is rather than swapped out for the nicer looking F1 number. N=3 or N=4 makes more sense as an actual deployment setting, since it trades away some recall for a much lower false positive rate on traffic the system has not seen before. The reset fix from Section 4.3 does help here, bringing the clean day false positive rate at N=2 down from 73.0% to 63.1%, though the underlying trade off between recall and false positives is still there either way.

5.3 Time-to-Onset Estimation

The countdown LSTM, trained on 249 real pre attack sequences taken from the CICIDS2017 timeline, gets a held out mean absolute error of about 374 seconds across 50 test sequences. An earlier version of this model trained on synthetic pre attack sequences had a much lower error, and that gap is exactly why the real data figure is the one used here: training on a generator answers an easier question than the one the system actually needs to answer on live traffic.

5.4 Fusion Layer Performance

Table 3 compares the combined fusion risk score against each of the three signals feeding into it, evaluated on their own over the same held out data.

Table 3: Fusion layer performance versus individual detection signals, held-out chronological split

Signal	Precision	Recall	F1-score	AUC
Isolation Forest score alone	0.493	0.537	0.514	0.763
CUSUM breach count alone	0.302	1.000	0.464	0.564
Random Forest confidence alone	0.335	0.970	0.498	0.930
Fusion (calibrated)	**0.719**	**0.955**	**0.821**	**0.963**

The fused score gets a higher ROC AUC than any of the three signals on their own, so the fusion layer is doing more than just echoing whichever input happens to be strongest. At the default cutoff, the fusion model's yes/no decision mostly lines up with the Random Forest's own attack flag, which makes sense since the Random Forest is the strongest single input going into it. The gain from fusing all three shows up mainly in how well the continuous risk score ranks things, not so much in where the hard yes/no line sits.

5.5 Adversarial Robustness

Table 4 shows the median relative perturbation, as a fraction of the distance to the benign centroid, needed to get the Random Forest to misclassify each attack type as benign.

Table 4: Median perturbation required for evasion, by attack category

Attack Category	Median Relative Perturbation
Botnet	0.37
Web Application	0.38
Brute Force	0.41
Reconnaissance	0.50
Denial of Service	0.54
Distributed Denial of Service	0.74

DDoS traffic is the hardest to disguise as benign, which makes sense given how extreme its volumetric signature is compared to normal traffic. Botnet is the easiest, since its low and slow beaconing pattern already sits fairly close to the benign centroid to begin with. It is worth pointing out that being able to evade the classifier in feature space does not mean a real attacker can pull it off: the ratio of downstream to upstream traffic, for example, is one of the features that would need to shift the most for evasion, but changing it would mean the attack's actual data flow has reversed direction, which defeats the point of running the attack at all. Each feature is checked against this kind of practical constraint rather than treating every number in the table as equally achievable.

5.6 Comparative Positioning

Table 5 lines PABD up against comparable approaches from the literature on the basis of what capabilities each one reports, not a claim that all of them were benchmarked on the same data, since no such shared evaluation exists across this literature to begin with.

Table 5: Comparative positioning of PABD against related approaches, by reported capability

Feature	**PABD (Proposed)**	**Signature-based IDS (e.g., Snort/Suricata)**	**Standalone ML Classifiers (Sharafaldin et al., 2018)**	**Unmitigated CUSUM (Tartakovsky et al., 2006)**
Detection timing	Pre-execution (drift-based)	Post-execution (signature match)	Post-execution (flow classification)	Pre-execution, but drift-only
Attack attribution	Random Forest, 7 real classes	Rule-based, no confidence score	Yes, no early-warning layer	Not applicable
Explainability	Live, per-alert SHAP	None	Typically none	Not applicable
Adversarial evaluation	Reported explicitly	Not typically reported	Not typically reported	Not typically reported
False-positive characterisation	Reported across operating points, including on unseen clean traffic	Widely reported as high in practice	Reported on tuning data only	High, without baseline adaptation

6 Conclusion and Future Work

This paper set out to build PABD, a network threat intelligence system that puts an Isolation Forest anomaly scorer, a reset corrected CUSUM drift detector, a Random Forest classifier and a real data trained LSTM countdown model behind one fusion layer, with live SHAP explanations and MITRE ATT&CK tags on top to make it auditable. Tested on real CICIDS2017 traffic across seven attack categories, the results show that watching for behavioural drift really can get ahead of signature based detection, that fusing the four signals ranks threats better than any one of them does alone, and that the classifier's ability to resist adversarial evasion, while not perfect, is limited in a real way by what an attacker can actually change without giving up on the attack itself. A few things were left out on purpose rather than papered over: there was no signature based IDS running alongside this system for a direct comparison, no second dataset was used to check whether these results hold up outside CICIDS2017, and the Lateral Movement and Infiltration classes simply do not have enough real examples in the data used here to be trusted, so neither is folded into the headline numbers.

There are three things worth doing next. Running a real signature based IDS side by side on the same traffic capture would let the lead time claim in this paper be measured directly instead of estimated against a flow fraction baseline. Testing against a second, independently collected dataset would show whether the detection and fusion numbers reported here hold up outside of CICIDS2017's particular way of generating traffic. And adding a causal discovery step to the drift layer, built around treatment and response timestamps specifically, could eventually make Lateral Movement detection possible once there is enough real ground truth data to build it on, which is not something the current dataset provides.

References

1. Sharafaldin, I., Lashkari, A. H., Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 108-116.
2. Panigrahi, R., Borah, S. (2018). A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems. *International Journal of Engineering & Technology*, 7(3.24), 479-482.

3. Liu, F. T., Ting, K. M., Zhou, Z.-H. (2008). Isolation Forest. *2008 Eighth IEEE International Conference on Data Mining*, 413-422.
4. Tartakovsky, A. G., Rozovskii, B. L., Blažek, R. B., Kim, H. (2006). Detection of intrusions in information systems by sequential change-point methods. *Statistical Methodology*, 3(3), 252-293.
5. Farkas, K. (2016). CUSUM anomaly detection. *M-Lab Technical Publications*.
6. Lundberg, S. M., Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
7. Ferrag, M. A., Maglaras, L., Moschoyiannis, S., Janicke, H. (2020). Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50, 102419.
8. MITRE Corporation. (2024). MITRE ATT&CK Framework. Retrieved from https://attack.mitre.org